

# Shared Memory and Concurrent Processes

Nafiul Khalid (PID: 6436457)  
COP 4610 Operating Systems Principles  
Florida International University  
Miami, Florida  
nkhal014@fiu.edu

**Abstract**—This project demonstrates the use of shared memory between multiple child processes. Four child processes access the same shared variable called `total` and increment it different numbers of times. The program was executed multiple times to observe how concurrent access to shared memory affects the final counter values. The results show that the values and process completion order change between executions because the shared variable is accessed without synchronization.

**Index Terms**—shared memory, processes, concurrency, race condition

## I. INTRODUCTION

The purpose of this project was to understand how multiple processes can communicate through shared memory and what can happen when several processes modify the same shared variable at the same time.

The program creates four child processes. All four processes share a variable called `total`. Process 1 increments the shared value 100,000 times, Process 2 increments it 200,000 times, Process 3 increments it 300,000 times, and Process 4 increments it 500,000 times.

The program was compiled and tested on the COP 4610 Ocelot virtual machine.

## II. IMPLEMENTATION

The program uses System V shared memory. A shared memory segment is created using `shmget()` and attached to the program using `shmat()`. The shared variable is initialized to zero before the child processes are created.

The four child processes are created using `fork()`. Each child calls its assigned process function and increments the shared value the required number of times.

The parent process uses `wait()` to wait until all four child processes finish. After the children finish, the parent detaches from the shared memory using `shmdt()` and removes the shared memory segment using `shmctl()`.

No synchronization method such as a semaphore or mutex was used. Therefore, multiple processes can read and modify the shared variable at the same time.

## III. EXPERIMENTAL RESULTS

The program was executed ten times on the COP 4610 Ocelot VM. The counter value printed by each process was recorded for every run.

The average, minimum, and maximum observed values are shown in Table II.

TABLE I  
COUNTER VALUES FROM TEN PROGRAM RUNS

| Run | P1     | P2     | P3     | P4     |
|-----|--------|--------|--------|--------|
| 1   | 115468 | 248588 | 389926 | 275468 |
| 2   | 168085 | 186536 | 239096 | 596677 |
| 3   | 108236 | 212214 | 375091 | 549640 |
| 4   | 104346 | 297876 | 319005 | 544641 |
| 5   | 135285 | 227534 | 171759 | 563672 |
| 6   | 105888 | 190256 | 436553 | 521639 |
| 7   | 86163  | 118422 | 343251 | 540917 |
| 8   | 96822  | 251319 | 358731 | 253612 |
| 9   | 51909  | 334808 | 292316 | 198546 |
| 10  | 111226 | 216119 | 355260 | 522893 |

TABLE II  
SUMMARY OF EXPERIMENTAL RESULTS

| Process | Average  | Minimum | Maximum |
|---------|----------|---------|---------|
| 1       | 108342.8 | 51909   | 168085  |
| 2       | 228367.2 | 118422  | 334808  |
| 3       | 328098.8 | 171759  | 436553  |
| 4       | 456770.5 | 198546  | 596677  |

## IV. ANALYSIS

The results were different in nearly every execution even though the same program and increment counts were used each time. This occurs because all four processes access the same shared variable without synchronization.

The statement that increments the value performs a read, an addition, and a write. If two or more processes perform these operations at about the same time, they may read the same old value and then overwrite each other's updates. This causes some increments to be lost.

The order in which the processes printed their results also changed between runs. For example, Process 4 sometimes printed before Process 3 or Process 2. This happens because the operating system scheduler determines when each process receives CPU time.

The value printed by a process also does not represent only the increments made by that process. Since all of the child processes use the same shared variable, the printed value includes changes that may have been made by other processes while the process was running.

These observations demonstrate a race condition. The final result depends on the timing and order in which the child processes access the shared variable.

## V. CONCLUSION

This project demonstrated how multiple processes can use shared memory to communicate. Four child processes successfully modified the same shared variable and the parent waited for all children before releasing the shared memory.

Running the program multiple times produced different counter values and different process completion orders. This showed that accessing shared memory without synchronization can produce unpredictable results because updates from different processes can interfere with each other.